



Tren

Tool Quick Reference

exiftool — read metadata from files

```
exiftool filename
```

file — identify file type by magic bytes

```
file suspicious.jpg
```

strings — extract printable strings from binaries

```
strings file.bin | grep -i flag
```

Wireshark — analyze packet captures; look for HTTP streams, suspicious payloads, follow TCP streams

Binwalk — find/extract embedded files

```
binwalk -e file.jpg
```

zsteg — detect steganography in PNG/BMP

```
zsteg image.png
```

unzip — extract archives; try

```
unzip -P password file.zip
```

for password-protected ones

Base64 — decode encoded strings

```
echo "SGVsbG8=" | base64 -d
```

General CTF flow to keep in mind:

1. `file`

first — never trust the extension

2. `exiftool`

for metadata flags

3. `strings`

for quick wins

4. `binwalk -e`

if something looks embedded

5. Check for Base64, hex, ROT13 in strings output

6. Wireshark: filter

`http`

or follow streams for hidden data

Go crush it — and make sure your teammate is there on time too!

Quick ways to decode binary:

Command Line (on the lab computers)

```
```bash
```

If it's a binary string like 01001000 01100101  
01101100 01101100 01101111

```
echo "01001000 01100101 01101100 01101100 01101111" | python3 -c "
import sys
data = sys.stdin.read().split()
```

```
print("".join(chr(int(b, 2)) for b in data))
```

```
"
```

```
...
```

### Or one-liner:

```
```bash
```

```
python3 -c "print(''.join(chr(int(x,2)) for x in '01001000 01100101'.split()))"
```

```
...
```

In Python (if you get shell access)

```
```python
```

```
binary = "01001000 01100101 01101100 01101100 01101111"
```

```
text = "".join(chr(int(b, 2)) for b in binary.split())
```

```
print(text) # Hello
```

```
...
```

---

### strings + grep combo (if binary is inside a file)

```
```bash
```

```
strings file.bin | grep -E "[01 ]+$"
```

```
...
```

Key things to check in CTF:

- Spaces between 8-bit groups → standard ASCII binary
 - No spaces → chunk into 8s first
 - Sometimes it's **binary** → **hex** → **ASCII** (two-step decode)
 - Could also be brainfuck or other esoteric encoding that *looks* like binary
-

The output

```
01001000 01100101 01101100 01101100 01101111
```

= **Hello** for reference.

Good luck in the lab!

Great thinking! Here's exactly what to do in the browser console during a CTF:

Open Console

F12

→ Console tab (or

Ctrl+Shift+J

)

Search for flags directly

```
```javascript
```

```
// Most CTF flags follow a pattern like FLAG{...} or CTF{...}
```

```
// Search entire page source
```

```
document.documentElement.innerHTML.match(/FLAG\{.*?\}/g)
```

```
document.documentElement.innerHTML.match(/CTF\{.*?\}/g)
```

```
// Try both lowercase too
```

```
document.documentElement.innerHTML.match(/flag\{.*?\}/gi)
```

```
```
```

Check HTML comments (very common hiding spot)

```
```javascript
```

```
// Get all comments
```

```
let walker = document.createTreeWalker(document, NodeFilter.SHOW_COMMENT);
```

```
let node;
while(node = walker.nextNode()) console.log(node.nodeValue);

...

```

---

## Check hidden elements

```
```javascript
// Hidden divs, spans etc
document.querySelectorAll('[hidden], [style="display:none"], [style="visibility:hidden"]')
.forEach(el => console.log(el.innerText, el.innerHTML))

...

```

Check page source directly

```
...

Ctrl+U → Ctrl+F → search "flag" or "CTF" or "{"

...

```

Check JavaScript files

```
...

F12 → Sources tab → look through .js files
Ctrl+Shift+F → search across ALL files for "flag"

...

```

Check cookies

```
```javascript
console.log(document.cookie)

...

```

---

## Check localStorage / sessionStorage

```
```javascript
console.log(JSON.stringify(localStorage))
console.log(JSON.stringify(sessionStorage))
```
```

---

## Check meta tags and headers

```
```javascript
// Meta tags
document.querySelectorAll('meta').forEach(m => console.log(m.name, m.content))

// Response headers → Network tab → click any request → Headers
```
```

---

## Check Base64 strings in source

```
```javascript
// Find potential base64 strings and decode them
let matches = document.documentElement.innerHTML.match(/[A-Za-z0-9+]{20,}={0,2}/g);
matches?.forEach(m => { try { console.log(atob(m)) } catch(e) {} })
```
```

---

## Priority order for CTF websites:

1. Ctrl+U
  - search "flag"
2. Console → innerHTML regex search
3. Sources tab →
  - Ctrl+Shift+F

global search

4. Check cookies, localStorage
  5. Network tab → response headers
  6. Hidden elements
  7. Decode any Base64 you find
- 

Since it's unencrypted HTTP, Wireshark can also catch the raw response — so run that alongside too!

Great combo strategy! Here's exactly how to use Wireshark on a specific website:

---

### **Step 1 — Start Capture**

...

Open Wireshark → select the network interface (usually eth0 or ens33 on lab computers)

Click the blue shark fin button to start

...

---

### **Step 2 — Filter for that specific website**

Since it's HTTP (unencrypted), use these filters:

...

## **By IP address (get IP first)**

`ip.addr == 192.168.x.x`

## **By hostname**

`http.host == "targetsite.com"`

## **Just all HTTP traffic**

`http`

# HTTP + the specific host combined

http.host contains "targetsite"

...

---

## Get the website's IP first

```
```bash
```

Run this in terminal before starting

```
ping targetsite.com
```

or

```
nslookup targetsite.com
```

...

Then filter by that IP in Wireshark

Step 3 — Trigger the traffic

...

Go to the website in browser

Refresh the page (Ctrl+R)

Click around on all pages

Submit any forms

...

Step 4 — Follow HTTP Stream (most important)

...

In Wireshark packet list:

Right click any HTTP packet → Follow → HTTP Stream

This shows you the FULL raw request + response

Search for flag{ or CTF{ in there

...

Step 5 — Key filters to find flags

...

See all HTTP responses

http.response

Filter by content type

http.content_type contains "text"

Search for specific string in packets

frame contains "flag"

frame contains "CTF"

frame contains "FLAG"

Find specific keyword

frame contains "{"

...

Step 6 — Export HTTP objects

...

File → Export Objects → HTTP

→ This downloads ALL files the website sent

→ Check each one for hidden flags

...

Step 7 — Check individual packet details

...

Click a packet → expand "Hypertext Transfer Protocol"

Look at:

- Request headers
- Response headers
- Response body (bottom panel shows raw data)

...

Full workflow visually:

...

Start capture

- Visit website
- Stop capture (Ctrl+E)
- Filter: http
- Right click packet → Follow HTTP Stream
- Ctrl+F → search "flag" or "CTF" or "{"
- Also check Export Objects

...

Pro tip for CTF:

Since it's a local lab network, also try:

...

Capture everything on the subnet

`ip.addr == 192.168.0.0/16`

You might catch OTHER teams' traffic too 🙈🙈

...

You've got a solid strategy — console + Wireshark together covers almost everything on an unencrypted site!

Yeah, here are more advanced techniques you might not think of during the pressure of a CTF:

Steganography on Website Images

```
```bash
```

## Download images from the site first

```
wget http://targetsite.com/image.jpg
```

## Then run all steg tools

```
exiftool image.jpg # metadata
strings image.jpg # hidden text
zsteg image.png # LSB steganography
binwalk -e image.jpg # embedded files
file image.jpg # verify file type
```
```

Directory Bruteforce (Gobuster style)

```
```bash
```

## Common hidden paths to manually check

```
/robots.txt
/sitemap.xml
/.htaccess
/backup.zip
/flag.txt
/secret.txt
/admin
/hidden
```

/notes.txt

/.git

...

Just type these in the browser URL bar one by one

---



## View Page Source Tricks

...

Ctrl+U → Full source

Look for:

- HTML comments <!-- flag here -->
- Weird whitespace (whitespace steganography)
- Text same color as background (white text on white)
- Zero width characters between normal text

...

---



## Cookie Manipulation

```
```javascript
```

```
// Read all cookies
```

```
console.log(document.cookie)
```

```
// Modify a cookie value
```

```
document.cookie = "admin=true"
```

```
document.cookie = "role=admin"
```

```
document.cookie = "auth=1"
```

```
// Then refresh and see if page changes
```

```
```
```

---



## Network Tab (DevTools)

...

F12 → Network tab → Refresh page

Click EVERY request and check:

- Response headers (flags hidden here often)
- Response body
- Request headers
- Preview tab

Especially check:

- XHR/Fetch requests
- JS files
- CSS files (yes, flags in CSS comments)

...

---

## JavaScript Reverse Engineering

```
```javascript
// In console - find all JS variables
Object.keys(window)

// Look for suspicious global variables
window.flag
window.secret
window.key
window.password

// Check all script tags
document.querySelectorAll('script')
.forEach(s => console.log(s.innerHTML))
```
```

---

## Common Encoding Chains in CTF

...

Binary → ASCII  
 Hex → ASCII  
 Base64 → decode  
 ROT13 → shift

URL encoded → %XX decode

HTML entities → & etc

## Decode URL encoding in console

```
decodeURIComponent("%46%4C%41%47")
```

## Decode HTML entities

```
let d = document.createElement('div')
d.innerHTML = "&FLAG&"
console.log(d.innerText)
...

```



### Check ALL File Downloads

```
```bash
```

If the site lets you download anything

```
file download.jpg # might not be an image
binwalk -e download.jpg # extract hidden zip/rar
unzip download.jpg # sometimes literally a zip
strings download.jpg | grep -i flag
...

```



Form Manipulation

```
```javascript
// Change hidden form fields
document.querySelectorAll('input[type="hidden"]')
.forEach(i => console.log(i.name, i.value))

// Modify and resubmit
document.querySelector('input[type="hidden"]').value = "admin"
...

```

---

## Check HTTP Methods

```
```bash
```

In terminal - try different HTTP methods

```
curl -X OPTIONS http://targetsite.com
curl -X PUT http://targetsite.com/flag
curl -v http://targetsite.com # -v shows full headers
curl http://targetsite.com/robots.txt
```
```

---

## Whitespace / Zero Width Steganography

```
```javascript
// Copy all text from page and check for zero-width chars
let text = document.body.innerText
let suspicious = [...text].filter(c => c.charCodeAt(0) < 32 ||
[0x200B, 0x200C, 0x200D, 0xFEFF].includes(c.charCodeAt(0)))
console.log(suspicious)
```
```

---

## Priority Checklist for Website CTF:

```
```
```

- ✓ Ctrl+U → search flag/CTF/{
- ✓ F12 Console → innerHTML regex
- ✓ /robots.txt and /sitemap.xml
- ✓ Network tab → all responses + headers
- ✓ Cookies + localStorage
- ✓ Hidden elements
- ✓ Download all images → steg tools
- ✓ JS global variables
- ✓ Follow HTTP stream in Wireshark
- ✓ Export HTTP objects in Wireshark

✓ Try /flag.txt /secret.txt /backup.zip

✓ curl -v for raw headers

✓ Check CSS files for comments

✓ Form hidden fields

...

You're going in really well prepared. Cover these systematically and you'll find it! 🏆